

A Matter of Perspective: Case Notion Selection for Object-Centric Performance Analysis (Extended Version)

Jan Niklas van Detten^{1,2} ✉, Pol Schumacher², and Sander J.J. Leemans¹

¹ RWTH University, Aachen, Germany

² Celonis, Munich, Germany

Abstract. Enterprise information systems track the interactions between business objects, which can be extracted into object-centric event logs. Object-centric process mining aims to analyze these logs to ideally improve business-relevant performance measures. This analysis requires a clear notion of which objects belong to a joint process execution, commonly called a case. Each such distinct case notion intuitively corresponds to one of many analytical perspectives on the underlying business process. To produce relevant analytical results, such a case notion needs to be carefully chosen. This poses a significant challenge in practice due to a lack of automated methods and the exponential number of possible case notions to choose from. In this paper, we introduce a novel framework to automatically select case notions for the performance measure analysis on object-centric event logs. In particular, we show how to automatically identify case notions that maximize relevant properties of the performance measure value distribution on the input log. As a result, manual work by practitioners is reduced to selecting a performance measure of interest and a distribution property that indicates optimization potential in the domain context. We evaluate our approach on a range of public logs and find it to produce relevant insights in feasible runtime without relying on domain knowledge for the case notion selection.

1 Introduction

Real-life business processes are often monitored by enterprise information systems. Process mining extracts these information into so-called *event logs* to analyze business processes. Ideally, this analysis leads to an improvement in business-relevant *performance measures*, such as the time spent per execution.

Traditionally, each process execution in an event log, commonly called a *case*, referred to the activities of an isolated *object*. However, most real-life processes contain multiple interacting objects of different types. Analysis techniques that explicitly account for such object interactions are referred to as *object-centric* [1].

The consideration of object interactions offers a much closer depiction of reality than their isolated analysis. However, it also induces a choice between exponentially many different notions of what constitutes a case [4,10]. Any subset of interactions in the input log can be considered relevant for that purpose.

Conceptually, each case notion represents one of many different perspectives to group the observed behavior in the input log. In practice, it is of particular interest to identify case notions, that align with the goal of any given analysis. In the context of a performance measure analysis, this equates to case notions that reveal the most interesting value distribution of the respective measure.

Technically, we consider case notions interesting for a performance analysis, if they lead to value distributions that maximize *distribution properties* that indicate performance issues. Such properties are typically well-defined by the operating context. They include, for example, the *standard deviation* which measures the spread, the *kurtosis* which quantifies the tail-heaviness, and the *skewness* which evaluates the asymmetry of the value distribution. For processes that require predictability and stability, a high standard deviation indicates a performance issue. In contrast, high values for kurtosis and skewness are more problematic for risk-averse and fairness-sensitive settings respectively. Depending on the performance measure and the distribution property, drastically different case notions might be needed to reveal the most optimization potential for a log.

However, to the best of our knowledge there are no automated techniques available to guide practitioners in the case notion selection for the performance analysis in object-centric event logs. While some initial ideas for the automated selection of case notions exists [7, 10], no investigation has been done in the context of performance analysis. As a result, object-centric performance analysis relies on a potentially suboptimal manual selection of predefined case notions.

In this paper we introduce a novel framework to automatically select case notions for a performance measure analysis on object-centric event logs. For a given object-centric event log, performance measure and distribution property, our approach automatically identifies a case notion that maximizes the selected property in polynomial time complexity. The identified case notion represents the most performance-relevant conceptual perspective on the underlying business process for the specified analysis setting. As a result, we eliminate the reliance on a manual case notion selection by domain experts for the object-centric performance measure analysis. Figure 1 illustrates the intended application of our approach for the performance analysis on object-centric event logs. We evaluate our approach on a range of public logs and find it to produce relevant insights in feasible runtime that outperform a brute force selection over existing case notions, without requiring manual work from experts for the case notion selection.

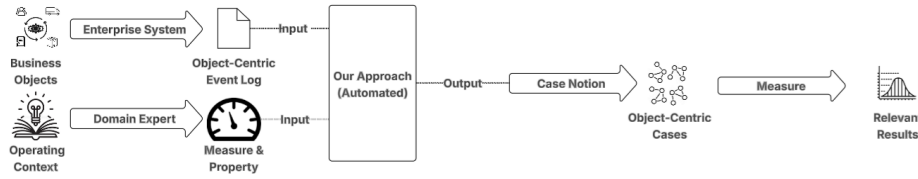


Fig. 1. Intended setting of our approach. For a given performance measure and distribution property, the optimal case notion is automatically identified by our approach.

2 Preliminaries

In this section we formalize object-centric event logs, case notions and performance measures. We write $\{\dots\}$, $\mathbb{P}(\{\dots\})$ and $\langle \dots \rangle$ for sets, power sets and sequences. We denote the universe of timestamps, events, activities, objects, object types, attributes and attribute values as $\mathbb{U}_{tim}, \mathbb{U}_{eve}, \mathbb{U}_{act}, \mathbb{U}_{obj}, \mathbb{U}_{typ}, \mathbb{U}_{att}, \mathbb{U}_{val}$.

2.1 Object-Centric Event Logs

An object-centric event log is a collection of events and objects with relations between them. Each event denotes the execution of an activity at a certain point in time and each object represents a business entity of a certain type. Relations between events and objects denote the involvement of the object in the execution of the event. Relations between objects and objects express a connection between them that is not necessarily reflected in the execution of any shared event. Attributes can be used to attach additional information.

Definition 1 (Object-Centric Event Log). *An object-centric event log $L = (E, O, E2O, O2O, \pi_{act}, \pi_{typ}, \pi_{tim}, \pi_{att})$ consists of events $E \subset \mathbb{U}_{eve}$, objects $O \subset \mathbb{U}_{obj}$, the relations between them $E2O \subseteq E \times O$, $O2O \subseteq O \times O$ and the projection functions for timestamps $\pi_{tim} : E \mapsto \mathbb{U}_{tim}$, activities $\pi_{act} : E \mapsto \mathbb{U}_{act}$, object types $\pi_{typ} : O \mapsto \mathbb{U}_{typ}$ and attribute values $\pi_{att} : (E \cup O) \times \mathbb{U}_{att} \mapsto \mathbb{U}_{val}$.*

Additionally, we write Ω and Σ for the set of object types and activity labels that are present in a given object-centric event log L respectively. Figure 2 shows a simplified log excerpt for an order management process. It tracks the behavior of customers, orders, packages, sales representatives and delivery trucks.

A case in an object-centric event log is a subset of objects and events in a log with the corresponding relations between them that are considered an independent execution of the business process for further analysis steps.

Definition 2 (Case). *A case in an object-centric log $L = (E, O, E2O, O2O, \pi_{act}, \pi_{typ}, \pi_{tim}, \pi_{att})$ is a tuple $(E', O', E2O', O2O', \pi'_{act}, \pi'_{typ}, \pi'_{tim}, \pi'_{att})$ with the events $E' \subseteq E$, the objects $O' \subseteq O$ and the relations $E2O' = E2O \cap E' \times O'$,*

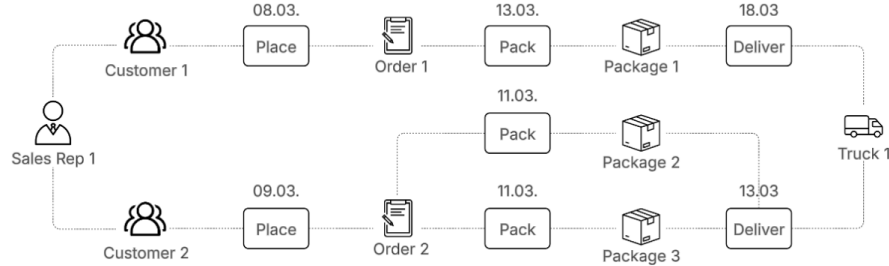


Fig. 2. Example object-centric event log excerpt for an order management process.

$O2O' = O2O \cap O' \times O'$. The projections $\pi'_{act}, \pi'_{typ}, \pi'_{tim}, \pi'_{att}$ are the same as $\pi_{act}, \pi_{typ}, \pi_{tim}, \pi_{att}$, but with domains restricted to the event and object subset.

Any subset combination of events and objects, with the corresponding relations between them, from the example log in Figure 2 can be considered a case. We write $c(L)$ for the set of all possible cases in an object-centric event log L . A function that projects an object-centric event log onto a set of its cases is called a *case notion*. We do not impose any restrictions on the structure of the resulting cases. That is, events and objects might be duplicated across multiple cases or missing from all of them, entirely depending on the selected case notion.

Definition 3 (Case Notion). Let $\mathbb{U}_L$ be the universe of object-centric event logs. A case notion, is any function $cn : \mathbb{U}_L \mapsto \mathbb{P}(\cup_{L \in \mathbb{U}_L} c(L))$ with $cn(L) \subseteq c(L)$.

The *traditional*, single object, case notion considers each object of a selected type and the associated events as its own case. Applying it to our example log from Figure 2 for the object type of orders or packages respectively results in the cases shown in Figure 3. Note how this leads to the duplication of some events and the complete removal of some objects. The *connected component* case notion considers all objects to be in the same case if they share any transitive relation [4]. For our example log, this notion only results in one case that covers the entire log. The *advanced* notion proposed in [10] constitutes a hybrid between the traditional and connected component notion by filtering certain relations.

2.2 Performance Measures

For the purpose of this paper, a performance measure is a function that maps each case to a numerical value. This number indicates the extent to which business value is provided by the case with regards to a quality dimensions [18].

Definition 4 (Performance Measure). A performance measure p for an object-centric event logs L maps each case to a numerical value: $p : c(L) \mapsto \mathbb{R}$.

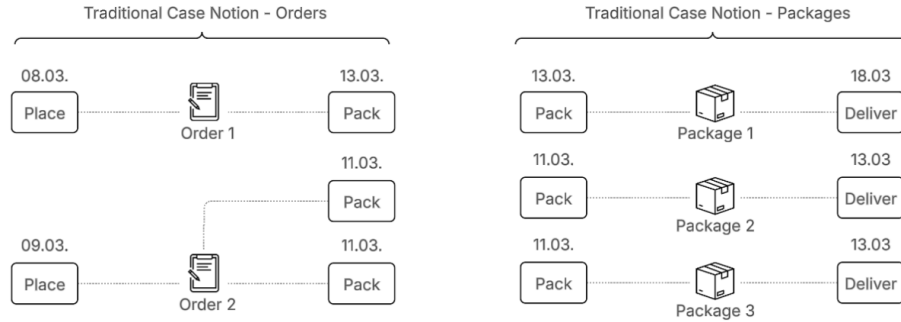


Fig. 3. Case resulting from the traditional case notion for orders and packages.

For most performance measures, there are multiple ways to define them in the object-centric setting. For our purpose, we shift this problem to the case notion. That is, given a case notion, we assume *all* elements per case to be relevant.

Following that line of thought, we define a few example performance measures. The *cycle time* for any given $case = (E', O', E2O', O2O', \pi'_{act}, \pi'_{typ}, \pi'_{tim}, \pi'_{att})$ refers to the time between the first and last event (1). The *resource usage* denotes the number of interactions per case that refer to objects of a specified resource type $ot \in \mathbb{U}_{ot}$ like a material or machine (2). The *total costs* per case can be calculated if corresponding information are provided in a numerical attribute $att \in \mathbb{U}_{att}$ as the corresponding sum across all involved objects and events (3).

$$p_{time}(case) = \max_{e \in E'}(\pi'_{tim}(e)) - \min_{e \in E'}(\pi'_{tim}(e)) \quad (1)$$

$$p_{usage}(case) = |\{(e, o) \in E' \mid \pi'_{typ}(o) = ot\}| \quad (2)$$

$$p_{cost}(case) = \sum_{o \in O'} \pi'_{att}(o, att) + \sum_{e \in E'} \pi'_{att}(e, att) \quad (3)$$

For further analysis, the results of a performance measure $p(case)$ are aggregated across the cases in a case notion $cn(L)$. This allows an investigation of various distribution properties. In this paper, we utilize the standard deviation (5), the skewness (6) and the kurtosis (7). Those properties measure the spread, the asymmetry and the tail-heaviness of the value distribution. High values for these properties indicate performance issues in different domain contexts.

$$\bar{x}(p, cn, L) = \sum_{case \in cn(L)} p(case) \cdot |cn(L)|^{-1} \quad (4)$$

$$\sigma(p, cn, L) = \sqrt{\sum_{case \in cn(L)} ((\bar{x}(p, cn, L) - p(case))^2 \cdot |cn(L)|^{-1})} \quad (5)$$

$$\gamma_1(p, cn, L) = \sum_{case \in cn(L)} ((\frac{\bar{x}(p, cn, L) - p(case)}{\sigma(p, cn, L)})^3 \cdot |cn(L)|^{-1}) \quad (6)$$

$$\gamma_2(p, cn, L) = \sum_{case \in cn(L)} ((\frac{\bar{x}(p, cn, L) - p(case)}{\sigma(p, cn, L)})^4 \cdot |cn(L)|^{-1}) - 3 \quad (7)$$

For our example log, we might be interested in checking the standard deviation of the cycle time performance measure, as a high value of that property would indicate reliability issues in the specific domain context. However, the value for this property differs drastically depending on the selected case notion. For the connected component case notion that results in a single case for the entire log as shown in Figure 2, the property value collapses to zero. The traditional case notions, as shown in Figure 3, avoid this issue but omit a lot of relations without evaluating their performance relevance. A brute-force investigation of all hybrid notions, including those proposed in [10], reveals that treating each order and its associated packages as a case leads to the highest standard deviation, indicating that there is a correlation between slow orders and slow packages. However, there are no automated techniques available to guide practitioners in the identification of such case notions for the performance measure analysis.

3 Performance Measure Analysis Framework

In this section we introduce a framework for the performance-specific object-centric case notion selection. As input, we expect an object-centric event log, a performance measure and a distribution property of interest. Then, we automatically identify a case notion that maximizes the selected distribution property.

For this purpose, we introduce an abstracted specification for case notions that allows us to define them based on the considered relations between objects and events. Then, we formalize an optimization problem to retrieve the case notion specification that leads to a performance measure distribution with a maximal value for the selected property of interest. Lastly, we propose a strategy to approximately solve this problem in polynomial runtime complexity.

3.1 Case Notion Specification

We introduce an abstract specification for object-centric case notions that defines how the resulting cases are constructed. Conceptually, this specification determines which objects are used to initialize a new case and which relations are transitively considered to include other objects and events in the same case. To ensure the consistency of the resulting cases, we abstract to the level of object types and activities. This allows us to reason about case notions based on the type of event-to-object and object-to-object relations they allow and forbid.

Formally, a case notion specification includes a set of starting object types and a set of relation types between object types and activities. The starting object types defines which objects are used to initialize a new case. The selected relation types describe which event-to-object and object-to-object relations are subsequently considered to transitively add events and objects to the case.

Definition 5 (Case Notion Specification). *A case notion specification ($start, rel$) is a tuple with a starting type set $start \subseteq \mathbb{U}_{typ}$ and a set of relation types between object types and activities $rel \subseteq \mathbb{U}_{typ} \times \mathbb{U}_{typ} \cup \mathbb{U}_{act} \times \mathbb{U}_{typ} \cup \mathbb{U}_{typ} \times \mathbb{U}_{act}$.*

Figure 4 shows all relevant relation types that can be included in a case notion specification for our example log. Additionally, an example specification is shown that considers the placement of an order and the packing and delivery of all associated packages as a case. Note that this abstracts away from the instance level of specific objects and events, as shown for the log in Figure 2. Additionally, as we use the included relation types to extract cases from the input log, the relation direction is relevant for potential transitive propagation.

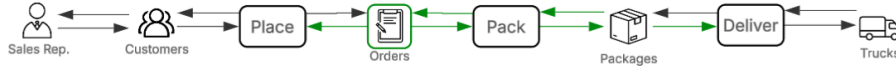


Fig. 4. Potential relationships for the case notion specification (entire graph) and an example specification with the order as the starting point and a few relations (green).

For a given object-centric log $L = (E, O, E2O, O2O, \pi_{act}, \pi_{typ}, \pi_{tim}, \pi_{att})$ and case notion specification $spec = (start, rel)$, cases can be iteratively extracted as follows. A new case $O' = \{o\}$ is initialized by a random object $o \in O$ of the starting object type $\pi_{typ}(o) \in start$ that has not yet been assigned to any case. The events associated to the new case are initially empty $E' = \{\}$. Then, objects and events are iteratively added to the case, if they are connected to any object or event that is already in the case by a relation that is included in the specification. First, we add all objects that share an allowed object-to-object relation with any object in the case (8) or an event-to-object relation with any event in the case (9). Then, we add all events that share an event-to-object relation with any object that is already included in the case (10).

$$O'_n = O' \cup \{o \in O \mid \exists o' \in O' : (o', o) \in O2O \wedge (\pi_{typ}(o'), \pi_{typ}(o) \in rel)\} \quad (8)$$

$$\cup \{o \in O \mid \exists e' \in E' : (e', o) \in E2O \wedge (\pi_{act}(e'), \pi_{typ}(o) \in rel)\} \quad (9)$$

$$E'_n = E' \cup \{e \in E \mid \exists o' \in O' : (o', e) \in E2O \wedge (\pi_{typ}(o'), \pi_{act}(e) \in rel)\} \quad (10)$$

We repeat these steps until no new object or event is added to the case. The object-to-object and event-to-object relations in the case are subsequently constructed in accordance with Definition 2. Then, the projections for object types, activity labels, timestamps, and attributes are added correspondingly, resulting in a finished case for the case notion specification. New cases are constructed until all objects of the starting object type are assigned. We write $cn_{spec}(L)$ for the resulting cases. Figure 5 shows the constructed cases based on the specification in Figure 4. Note that due to the direction of the relations, packages that are jointly delivered would not be in the same case if they refer to different orders.

Importantly, all case notions proposed in literature so far can be represented by our specification. The connected component case notion corresponds to all relations between object types and activities being included in a specification with all types being start types. The traditional single object case notion is specified by only having relations between the starting object type and all activities in the specification. Lastly, the advanced case notion can be expressed by using the specified starting type and by only allowing relations between object types and activities if they adhere to the classification criteria proposed in [11].

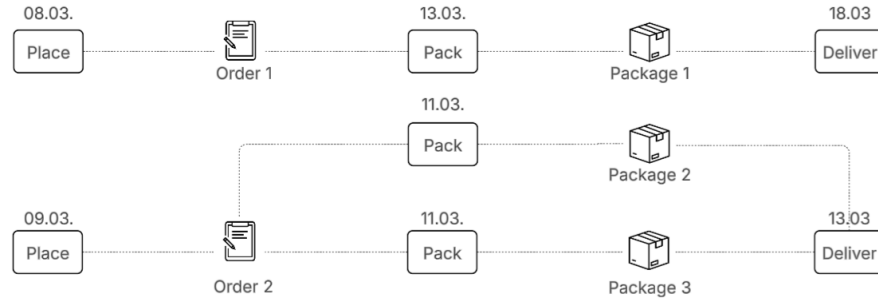


Fig. 5. Resulting cases for the example case notion specification in Figure 4.

3.2 Case Notion Optimization

Based upon the specification of object-centric case notions, we can formulate an optimization problem to identify case notions that lead to desired performance measure value distributions with regards to a specified distribution property of interest. Formally, for any given object-centric event log L and performance indicator $p(case)$, this is the solution of maximizing the selected distribution property (11) across all case notion specifications (12). Note that we depicted the optimization of the standard deviation here as an example, which can be replaced by any other property such as the skewness (6) or kurtosis (7).

$$\arg \max_{spec \in \mathbb{U}_{spec}} \sigma(p, cn_{spec}, L) \quad (11)$$

$$s.t. \quad \mathbb{U}_{spec} = \mathbb{P}(\Omega) \times \mathbb{P}(\Omega \times \Omega \cup \Sigma \times \Omega \cup \Omega \times \Sigma) \quad (12)$$

In case of enough computational resources or small input logs, this problem can be solved by brute force. However, as the number of potential case notion specifications increases exponentially with the number of object types and activities, we do not expect this to be feasible on any real-life object-centric log.

As such, we introduce a strategy to approximate the solution of the optimization problem from above in polynomial runtime. For this purpose, we replace the full optimization problem with a sequence of smaller optimization problems that are easier to solve. This strategy only provides local, and not global, optimality guarantee but reduces the complexity from exponential to polynomial.

Conceptually, we consider all potential relations between object types and activities as edges in a graph, as shown in Figure 4. Each node in that graph is considered a potential starting point for our approximation strategy. For each starting point, we measure the value of the selected distribution property for the case notion specification that includes all outgoing edges of that node. Then, we iteratively add relations that correspond to the outgoing edges of the neighboring

Algorithm 1 Case Notion Optimization (Standard Deviation)

```

function optimize( $L, p$ )
   $property, start, rel = 0, \Omega, (\Sigma \times \Omega) \cup (\Omega \times \Sigma) \cup (\Omega \times \Omega)$ 
  for  $ot \in \Omega$  do
     $start', rel' \leftarrow \{ot\}, \{ot\} \times \Sigma$ 
     $property' \leftarrow \sigma(p, cn_{start', rel'}, L)$ 
    while true do
       $node \leftarrow \arg \max_{n \in \Sigma \cup \Omega} \sigma(p, cn_{start', rel' \cup (\{n\} \times \Omega) \cup (\{n\} \times \Sigma)}, L)$ 
      if  $\sigma(p, cn_{start', rel' \cup \{node\} \times \Omega \cup \{node\} \times \Sigma}, L) - property' \leq 0$  then
        break
       $rel' \leftarrow rel' \cup \{node\} \times \Omega \cup \{node\} \times \Sigma$ 
       $property' \leftarrow \sigma(p, cn_{start', rel'}, L)$ 
    if  $property' > property$  then
       $property, start, rel \leftarrow deviation', start', rel'$ 
  return ( $start, rel$ )

```

node that lead to the highest increase in distribution property. If no further improvement can be found, the case notion specification is considered done. Finally, we pick the optimal notion across all starting nodes. Note that within our approximation, we do not perform a subset selection per node, as this would induce an exponential complexity. That is, a node can only be fully expanded or not, including or excluding all outgoing relations of that node.

Algorithm 1 shows this approximation strategy, with $\sigma(p, cn, L)$ denoting standard deviation. As before, this can be exchanged for other distribution properties. Applying the approach to the cycle time standard deviation on our object-centric example log, results in the following iteration steps for the starting node of the object type order, illustrated in Figure 6. First, the starting node of the object type order is initialized by including all relations that correspond to outgoing edges in the case notion specification (green). Then, for each neighboring node, i.e. the pack and place activity, it is checked if expanding the specification with the relations of the outgoing edges increases the standard deviation of the performance indicator. This is only the case for the outgoing edges of the packing activity node. As such, the corresponding relations (blue) that lead to an increase in standard deviation are included in the case notion specification. In the subsequent iteration, the object type node of packages is analogously expanded (yellow). Afterwards, no further improvement can be found and the case notion specification is considered complete. The resulting cases can be seen in Figure 5. They exhibit a higher standard deviation than any of the predefined case notions do on the same log. From an analytical perspective, this indicates that the combination of packages and orders offers an insight. In particular, we can see that higher cycle times correlate between connected orders and packages, indicating that a delay in the order might cause a delay in the package.

The resulting case notion specification will always be detected in polynomial runtime complexity. The number of potential nodes to check is limited by the number of object types and activities and decreases by one in each iteration. Within each iteration only the remaining nodes need to be checked, resulting in a complexity of $(|\Omega| + |\Sigma|)^2$ per starting object type $|\Omega|$. Notice that in case of the brute force approach referred to in the optimization problem formulation above, all combinations of relations are investigated per starting object type, i.e. the worst case time complexity is exponentially limited by $|\Omega| \cdot 2^{(|\Omega|^2 + |\Omega| \cdot |\Sigma|)}$.

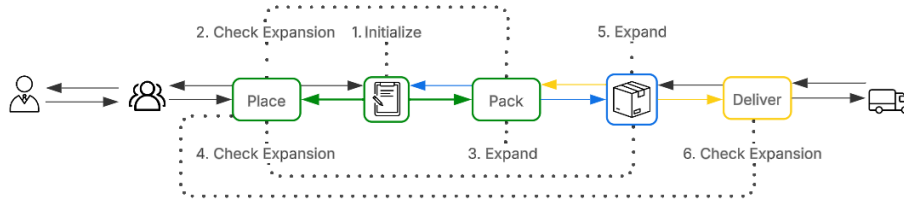


Fig. 6. Application of our approximation on the example log, starting from orders.

4 Evaluation

In this section we evaluate our approach threefold. First, we measure runtime demands on public object-centric logs to evaluate the feasibility of our approach. Secondly, we compare the distribution property values of the case notions identified by our approach to those of existing case notions from [4, 10] to quantify the advantage over a manual selection. Thirdly, we perform an example application to showcase the analytical value of the automatically generated results. Lastly, we discuss the experimental results and relevant limitations of our approach.

4.1 Runtime Demands

We provide a public implementation of our approach (<https://github.com/Nik314/CaseNotionPerformance>) and apply it to ten publicly available object-centric event logs from the library of the OCEL file format.

We use the cycle time, resource usage and total costs as the performance measures and combine them with the three distribution properties of standard deviation, skewness and kurtosis. As such, there are nine potential combinations of performance measures and distribution properties per log. The cycle time does not require any additional input and can hence be applied to all ten logs. The resource usage can be applied to all ten logs as well, as they contain suitable object types. For the total cost, we only found suitable attributes in five of the investigated logs and omitted the application of this measure on the rest.

For each of the resulting 75 suitable combinations of logs, performance measures and distribution properties, we measure the runtime required for the identification of a case notion specification on a consumer laptop with an Intel Core i9-13980HX processor and 64GB RAM. The statistics of the investigated input logs and the aggregated runtime results of our approach are shown in Table 1.

Across all setups, on average, the approximation of a case notion specification by our approach took 735 seconds per object-centric input log with a wide

Table 1. Statistics of utilized object-centric logs and measured runtime, aggregated across all combinations of performance measures and distribution properties per log.

ID	Log	Objects	Events	Activities	Types	Runtime		
						min	mean	max
01	P2P	9054	14671	10	7	141s	382s	624s
02	Recruiting	1505	6980	16	6	31s	133s	400s
03	LRMS O2C	8819	28278	22	9	807s	1604s	2705s
04	LRMS P2P	4493	8300	12	6	51s	102s	149s
05	LRMS HR	3768	18119	24	6	701s	1280s	1962s
06	Hospital	3688	14642	9	8	81s	164s	245s
07	Github	532	1798	20	3	7s	12s	14s
08	Logistics	13882	35372	14	7	368	1196s	2323s
09	Production	23771	38528	11	12	886s	2256s	3159s
10	Transfer	2500	10319	3	5	13s	41s	83s

range, spanning from 7 seconds to 3156 seconds. When averaging all setups per individual input log, the results indicate that the number of object types and activities, and the number of event-to-object and object-to-object relations influence the runtime the most. This was expected, since the number of object types and activities $(|\Omega| + |\Sigma|)^2$ limits the number of nodes to be investigated by our strategy, as discussed in the previous section. A higher number of event-to-object and object-to-object relations $(|E2O| + |O2O|)$ increases the number of operations needed for the extraction of the cases from any given case notion.

Additionally, we also aggregated the results by performance measure and distribution property respectively to evaluate their runtime relevance. For the logs investigated here, the distribution property had only negligible effects on the runtime. This was expected as the computational differences between them are negligible in terms of complexity. In contrast, the performance measures showed to exhibit an influence on the runtime with the cycle time leading to the fastest results and the resource usage and total cost to the slowest. We assume this to be caused by implementation-specific details as there should theoretically be no inherent complexity differences between the measures. Profiling confirmed that the utilized data structure causes overhead when accessing certain relation types and attributes in contrast to timestamps relevant for the cycle time.

4.2 Case Notion Comparison

Next, we compare our approach against a brute force selection from existing case notions proposed in literature so far. That is, we measure the property values for each combination of performance measure, distribution property and input log that result from testing out the traditional, connected component and advanced case notions. Then, we compare the results against the property values produced by the case notion specification identified by our approach. For the traditional and advanced case notion, a starting object type needs to be selected to initiate the case notion. Following brute force, we report the results for all start types. Note that our approach and the connected component case notion only induce a single point, as there is no need to choose a starting object type for them.

The results for three performance measure and distribution property combinations, i.e. the the cycle time standard deviation, the resource usage kurtosis and the total cost skewness, can be seen in Figure 7. All noteworthy high-level insights in them are consistent across the remaining six combinations, which we omitted here for space reasons and included them in the repository instead.

First, we can clearly observe that there are vast differences between the selected case notions on the investigated logs. This indicates the importance of an explicit case notion selection during the analysis. Additionally, we can see for the traditional and advanced case notion, that the choice of the starting object type has a large impact on the resulting values. For the traditional notion, this was expected as the resulting performance values refer to disjoint object sets. However, for the advanced case notions it is surprising as the filtering of event-to-object relations remains the same across all starting types [10]. On the investigated logs, the advanced case notion is sensitive to that selection on all of



Fig. 7. Distribution property values per log and case notion on a logarithmic scale. For the traditional and advanced notion, the plot shows results for all start types.

the logs. As such, a careful choice of the starting type seems to be a necessity as well. The connected component case notion does not suffer from this problem, as all types are considered as starting types. However, as already reported in [10], the connected component case notion can collapse into a single case due to high connectivity. In our experiment, this occurred on half of the logs.

When comparing the overall brute force optimum across the existing case notions against our automated case notion selection, a clear trend becomes apparent. In our investigation, across all logs, all performance measures and all distribution properties our approach identifies a case notion specification that results in an equal or higher value of the selected property of interest.

4.3 Example Application

To showcase the utility of the automatically generated results, we perform an in-depth analysis on one of the logs. For this purpose, we chose the cycle time

standard deviation on the recruiting process (02). We investigate the resulting case notion from our approach and compare the cases with the best and the worst performance to check if we can find any reason for performance issues.

The case notion identified on the full log uses the vacant positions and their corresponding activities as the starting point for new cases. In contrast to the connected component case notion, multiple relations are excluded. In particular the outgoing relations from the object type of hiring managers and the five activities relating to performing interviews and deciding on job offers are excluded. While these are only few deviations from the connected component case notion, they are important to not produce only a single case for the entire log. Given the complexity of the identified specification, we doubt that a human analyst could have identified this case notion manually on the investigated input log.

While the specification itself seems complex, the resulting cases reveal a clear picture. The resulting cases exhibit two distinct performance values that are drastically different. The best performing cases include all vacant positions for which an offer was quickly made and accepted. In contrast, all other cases include a vacant position for which either no offer was made or for which all offers were rejected. In these cases, the vacant positions were repeatedly assigned to other hiring managers until they were finally closed without hiring any candidate. Those closings probably represent automated actions, as all unfilled positions were closed exactly one year after their initial creation. As a result, these cases all have the same cycle time of a full year. The business relevance of this insight is clear. Cases in which the vacant position was not quickly filled, were shuffled between managers and finally left to be automatically canceled after a year. For future vacant positions, counter measures should be taken if a position cannot be filled quickly, instead of shifting it between managers without positive effect.

We proceed by removing all events and objects in these problematic cases from the log and repeat the analysis to see if further insights can be found in the remaining parts. The case notion identified in this second step refers to the applications as the starting object type. Further outgoing propagation to other activities are transitively exhaustive. However, outgoing relations from most other object types are mostly disregarded. In particular, relations from the vacant positions, the interviewers and the managers are not considered. This makes sense, as these types induce resource-like behavior in relation to the application. As a result, the resulting cases are much smaller than before and reflect the life cycle of an applicant that submits one or more application, resulting in no, one, or multiple offers. Sorting these cases by their performance value reveals applicants that apply separately for multiple positions at the bottom of the performance spectrum. The reason seems to be clear, as the entire application process is repeated, even when applying to vacant positions of the same kind. Here, it seems beneficial to reuse insights from previous applications if possible.

We end the analysis at this point for space reasons. In principle one could repeat this process of 1) automatically identifying a case notion, 2) investigating the most critical cases, and 3) finally removing them from the input log until one arrives conceptually at the level of one of the traditional case notions.

4.4 Discussion

Our evaluation showed that our approach identifies case notions that lead to performance value distributions with high distribution property values of interest in feasible runtime. Across all logs, performance measures and distribution properties, the observed results of the identified case notions equaled or exceeded those of a brute force optimum across all existing case notions proposed in literature so far. For the log further investigated in our example application, the generated insights showed analytical value at different levels of analysis. Conceptually, our approach allowed us to traverse the spectrum between the fully connected and the traditional case notions to generate insights at different levels.

Our approach remains limited by a few design choices and experimental restrictions. We intentionally enforce consistency on the inclusion of relations between objects and events on the type-activity level. We consider this a reasonable choice given that object-centric event logs are typically extracted from relational databases in which similar consistency requirements are common. However, we cannot rule out that even better case notions could be found that differentiate relations on the level of individual object and event instances. The same remark is true for the selection of relation subsets per object type or activity node, which we omit from our approximation strategy for the sake of runtime feasibility.

Furthermore, it should be explicitly noted that the input requirement of a distribution property that indicates optimization potential in the domain context remains as a domain knowledge dependency for our approach. Additionally, while the results of our evaluation consistently showed our approach to achieve its designated goal, we cannot generalize corresponding claims to performance indicators and distributions properties outside of the ones we investigated.

5 Related Work

We summarize existing work on object-centric case notions and performance analysis. For a general overview on object-centric process mining we refer to [8].

Formal definitions of cases have first been lifted into the object-centric in [6]. Extensions with case notions that include correctness guarantees have been proposed in [4]. Further investigations to reduce the complexity of the resulting cases while attempting to maintain most of the formal correctness have been performed in [10]. These approaches include some tooling to investigate the structure of the resulting cases, enabling visual performance analysis [3, 5, 16]. However, the approaches are tied to a process model that is able to replay the cases. As such, they rely on discovery techniques to construct process models [2, 9]. Additionally, they refer to non-adaptable predefined case notions.

Some definitions for performance indicators in object-centric event logs have been proposed in [13]. In contrast to our work, the approach from [13] does not consider arbitrary case notions and is limited to combinations of the single object case notion obtained by flattening the log on individual object types [1].

Alternative object-centric performance analysis strategies include the use of graph structures, that enrich the control flow with additional information, such

as the time spent [17]. The approach inherently aggregates performance values and deducts them from the observed control flow. Applying the approach without further preprocessing of the event log equates to the use of the connected component case notion proposed in [4]. It has been further extended in [17] and adjusted to operational monitoring needs in a live process mining setting.

Various other performance analysis techniques have been proposed before the introduction of object-centric process mining, such as [11]. Those primarily focus on isolated object types and hence equate to the use of the traditional case notion in the context of our work. Some exceptions include the approaches proposed in [12, 15], which perform an hierarchical and inter-organizational performance analysis on traditional logs. As such, one could argue that they reflect the implicit use of multiple predefined case notions for different levels of analysis. However, arbitrary case notions and corresponding performance indicator are not included.

The optimization of case notions towards certain properties has been investigated in some initial work outside the context of performance analysis. The approach in [10] provides guidance on the selection of case notions based on the complexity and correctness of the resulting cases. The proposal in [7] enriches case notions with additional information to create so-called views that can also include information on the involvement of objects. The selected views are subsequently optimized to maximize their representative power in the context of exploratory process analysis. Other approaches, such as [14] aim to directly embed relevant information on the scoping of object-centric processes in the log during extraction, which is comparable to case notions. However, this reintroduces reliance on domain knowledge and can as such not be used automatically.

To the best of our knowledge, our approach is the first to automatically navigate the landscape of arbitrary case notions for the performance analysis on object-centric logs without relying on predefined notions or domain knowledge.

6 Conclusion

In this paper, we introduced a framework to automatically identify case notions for the object-centric performance measure analysis. As such, we removed the dependency for domain experts to select case notions manually or by brute force. Additionally, we provided approximation methods that remain in polynomial complexity to enable an application on real-life logs. We evaluated our approach on a range of logs and found it to be feasible. The identified case notion clearly outperformed a brute force selection from existing case notions. This effect was observed without exceptions for multiple combinations of performance measures and distribution properties. Additionally, we showcased the analytical value of our approach on multiple levels of analysis in an example application. Our approach remains limited by a few intentional design choices that ensure runtime feasibility but might leave some room for further optimizations of the resulting case notions. We intend to quantify this gap in future work and provide more guidance on the distribution property selection per domain context.

References

1. van der Aalst, W.M.P.: Object-centric process mining: Dealing with divergence and convergence in event data (2019)
2. van der Aalst, W.M.P., Berti, A.: Discovering object-centric petri nets. *Fundam. Informaticae* **175**(1-4), 1–40 (2020)
3. Adams, J.N., Aalst, W.: OC: Object-Centric Process Insights, pp. 139–150 (2022)
4. Adams, J.N., van der Aalst, W.M.P.: Addressing convergence, divergence, and deficiency issues. In: *Business Process Management Workshops - BPM 2023 Workshops*. Lecture Notes in Business Information Processing, vol. 492. Springer (2023)
5. Adams, J.N., Park, G., van der Aalst, W.M.P.: ocpa: A python library for object-centric process analysis. *Softw. Impacts* **14**, 100438 (2022)
6. Adams, J.N., Schuster, D., Schmitz, S., Schuh, G., van der Aalst, W.M.P.: Defining cases and variants for object-centric event data. In: *4th International Conference on Process Mining, ICPM 2022*. pp. 128–135. IEEE (2022)
7. Basmer, M., Ueck, H., Fahland, D., Weidlich, M.: MANTA: materializing views on event data for context exploration in process analysis. In: *Business Process Management - 23rd International Conference, BPM 2025 Proceedings*. Lecture Notes in Computer Science, vol. 16044, pp. 51–68. Springer (2025)
8. Berti, A., Montali, M., van der Aalst, W.M.P.: Advancements and challenges in object-centric process mining: A systematic literature review (2023)
9. van Detten, J.N., Schumacher, P., Leemans, S.J.J.: Discovering compact, live and identifier-sound object-centric process models. In: *6th International Conference on Process Mining, ICPM 2024*. pp. 113–120. IEEE (2024)
10. van Detten, J.N., Schumacher, P., Leemans, S.J.J.: A framework for advanced case notions in object-centric process mining. In: *Process Mining Workshops*. pp. 402–414. Springer Nature Switzerland, Cham (2025)
11. van Dongen, B.F., Adriansyah, A.: Process mining: Fuzzy clustering and performance visualization. In: *Business Process Management Workshops, BPM 2009 International Workshops, Revised Papers*. Lecture Notes in Business Information Processing, vol. 43, pp. 158–169. Springer (2009)
12. Engel, R., Krathu, W., Zapletal, M., Pichler, C., Bose, R.P.J.C., van der Aalst, W.M.P., Werthner, H., Huemer, C.: Analyzing inter-organizational business processes - process mining and business performance analysis using electronic data interchange messages. *Inf. Syst. E Bus. Manag.* **14**(3), 577–612 (2016)
13. Estrada-Torres, B., del Río-Ortega, A., Resinas, M.: Defining process performance measures in an object-centric context. In: *Business Process Management Workshops*. pp. 210–222. Springer International Publishing, Cham (2023)
14. Khayatbashi, S., Rafiei, M., Chen, J., Kampik, T., Berg, G., Jalali, A.: Enriching object-centric event data with process scopes: A framework for aggregation and analysis. *CoRR* (2025)
15. Leemans, M., van der Aalst, W.M.P., van den Brand, M.G.J.: Hierarchical performance analysis for process mining. In: *Proceedings of the 2018 International Conference on Software and System Process*. pp. 96–105. ACM (2018)
16. Park, G., Adams, J.N., Aalst, W.: OPerA: Object-Centric Performance Analysis
17. Park, G., Adams, J.N., van der Aalst, W.M.P.: Conformance checking and performance analysis using object-centric directly-follows graphs. In: *Business Process Management Forum*. pp. 179–196. Springer Nature Switzerland, Cham (2024)
18. del Río-Ortega, A., Resinas, M., Cabanillas, C., Ruiz, A.: On the definition and design-time analysis of process performance indicators. *Information Systems* (2013)